Session 03 — Teach Sheet

Problem Solving & Debugging

Programming Hero

Theme: Assignment Prep — Bringing Everything Together

16 Problems	8 Debug Bugs	1 Framework
Mixing every topic so far	Syntax · Runtime · Logical	UPER problem-solving method

How to Use This Document

1	This session has NO new syntax — it's pure practice, combining everything from Sessions 1 & 2.
2	Start with the Problem-Solving Framework (UPER) — this is the mindset for the whole session.
3	Solve the A-type problems live — narrate your thinking out loud, not just the code.
4	Give students 5–7 minutes per B-type problem — these mirror real assignment questions.
5	For debugging cards, resist fixing it for students immediately — let them stare at the bug first.
6	Share the Answer Sheet at the end — it also has a 'Common Bugs to Avoid' cheat sheet.

Part 1 — The Problem-Solving Framework

Before writing a single line of code, every strong problem-solver follows the same four steps. This is called UPER — Understand, Plan, Execute, Review. Skipping straight to typing code is the #1 reason students get stuck on assignments.

> 🧭 *Think of UPER like reading a math word problem. You don't grab a calculator immediately — you read the question twice, figure out what's being asked, decide which formula applies, THEN calculate, and finally check if the answer makes sense.*

Step	What to do
U — Understand	Read the problem twice. What is the INPUT? What is the expected OUTPUT?
P — Plan	Before coding, write the steps in plain English or on paper. Which loop? Which condition?
E — Execute	Now write the code — translate your plan into JavaScript, one step at a time.
R — Review	Test with different inputs. Does it still work for edge cases (0, negative, empty array)?

Worked Example — Largest of Three Numbers

U — Understand: Given three numbers, find which one is the biggest.
P — Plan: Compare a vs b vs c using if-else — whichever is bigger than both others is the answer.
E — Execute: Write the if-else chain (see code below).
R — Review: Test with (5,5,5) — a tie. Test with negative numbers. Does it still work?

```
function findLargest(a, b, c) {
  if (a >= b && a >= c) return a;
  if (b >= a && b >= c) return b;
  return c;
}

console.log(findLargest(12, 45, 7));  // 45
```

Part 2 — Problem Solving Practice

These problems deliberately mix concepts from every session so far — numbers, strings, arrays, objects, and functions. This is exactly the kind of combination your assignment will expect.

A-01 — Largest & Smallest in an Array 🟢 Easy

Concepts Used: *Arrays · Loops · Conditionals*

Scenario: Given an array of numbers, find both the largest AND smallest value using a single loop (no built-in Math.max/Math.min).

- If the input is not an array, return "Invalid".
- If any element of the array is not a number, return "Invalid".

Expected Output:

```
Numbers: [45, 12, 89, 3, 67]
Largest: 89
Smallest: 3
```

Hints:

1. Initialize both max and min to the first element
2. Update them inside one loop as you compare each number

Write your code here:

A-02 — Word Position Filter 🟡 Medium

Concepts Used: *Strings · Loops · Modulus*

Scenario: SnapText, a note-taking app, builds a quick preview of a caption by keeping only the words at even index positions (0, 2, 4...). Write a function filterEvenPositionWords(sentence) that returns the filtered sentence.

Expected Output:

```
filterEvenPositionWords('The quick brown fox jumps')
  → 'The brown jumps'
```

Hints:

1. split(' ') into an array of words
2. Loop through the words and keep the ones where the index passes i % 2 === 0

Write your code here:

A-03 — Weekly Steps Summary (done) 🟡 Medium

Concepts Used: *Functions · Loops · Objects*

Scenario: FitTrack, a fitness app, logs a user's daily step counts for the week in an array. Write a function weeklyStepsSummary(stepsArray) that returns an object { totalSteps, goalReached }, where goalReached is true once totalSteps reaches 50000.

- If stepsArray is not an array, return "Invalid".
- If any value inside the array is not a number, return "Invalid".

Expected Output:

```
weeklyStepsSummary([8000,7500,9200,6000,10000,5500,4000])
  → { totalSteps: 50200, goalReached: true }
```

Hints:

1. Loop through the array and add each day's steps to a running total
2. Compare the total to 50000 to set goalReached, then return both in one object

Write your code here:

A-04 — Palindrome Checker 🟡 Medium

Concepts Used: *Strings · Reversal*

Scenario: Write a function isPalindrome(str) that returns true if the string reads the same forwards and backwards (e.g. 'madam').

- If the input is not a string, return "Invalid".

Expected Output:

```
isPalindrome('madam') → true
isPalindrome('hero')  → false
```

Hints:

1. Reverse the string using any method from Session 2
2. Compare the reversed version to the original with ===

Write your code here:

A-05 — Count Vowels in a String (done) 🟢 Easy

Concepts Used: *Strings · Loops · Conditionals*

Scenario: Write a function countVowels(str) that counts how many vowels (a, e, i, o, u) appear in a string, case-insensitive.

- If the input is not a string, return "Invalid".

Expected Output:

```
countVowels('Hello World') → 3
```

Hints:

1. Lowercase the string first so case doesn't matter
2. Loop through each character and check if it's in 'aeiou' using includes()

Write your code here:

A-06 — Remove Duplicate Values 🟡 Medium

Concepts Used: *Arrays · Loops · includes()*

Scenario: Write a function removeDuplicates(arr) that returns a new array with only unique values, preserving order.

- If the input is not an array, return "Invalid".

Expected Output:

```
removeDuplicates([1,2,2,3,4,4,5]) → [1, 2, 3, 4, 5]
```

Hints:

1. Create an empty result array
2. For each item, only push it if the result array doesn't already includes() it

Write your code here:

A-07 — Student Report Card Generator 🟡 Medium

Concepts Used: *Objects · Functions · Conditionals*

Scenario: Given a single student object with name and three subject marks (bangla, english, math), write a function generateReportCard(student) that returns a NEW object containing the student's name, total, average, and grade (A+ for 90+, A for 80+, B for 70+, F below 70).

- Return "Invalid" if:
  - student is not an object
  - bangla is not a number
  - english is not a number

- math is not a number

Expected Output:

```
student = { name:'Ayan', bangla:78, english:85, math:92 }
Report: { name:'Ayan', total:255, average:85, grade:'A' }
```

Hints:

1. Access marks with student.bangla, student.english, student.math
2. Build and return a brand-new object — don't modify the original student object

Write your code here:

A-08 — Leap Year Checker 🟢 Easy

Concepts Used: *Functions · Logical Operators*

Scenario: Write a function isLeapYear(year) that returns true if the year is a leap year. Rule: divisible by 4 AND (not divisible by 100 OR divisible by 400).

- Return "Invalid" if the input isn't a number.

Expected Output:

```
isLeapYear(2024) → true
isLeapYear(1900) → false
isLeapYear(2000) → true
```

Hints:

1. Combine % checks with && and ||
2. Test all three given examples to confirm your logic

Write your code here:

A-09 — Email Domain Analyzer (done) 🟡 Medium

Concepts Used: *String · Split · Loop · Object*

Scenario: MailBox Pro, an email management system, wants to analyze a sentence containing email addresses. Write a function analyzeEmailDomains(text) that returns an object containing the total number of email addresses and the longest email domain (the part after @).

- "Invalid" if the input is not a string.

Expected Output:

- `analyzeEmailDomains("Contact support@gmail.com admin@yahoo.com info@programminghero.com")`
  `→ { emailCount: 3, longestDomain: "programminghero.com" }`

- `analyzeEmailDomains("Hello everyone!")`
  `→ { emailCount: 0, longestDomain: "" }`

Hints:

1. Use split(" ") to separate the sentence into words.
2. Check whether a word contains "@" using includes("@"), then split("@") to extract the domain and compare its length.

Write your code here:

B-01 — Sum of Digits 🟢 Easy

Concepts Used: *Loops · Modulus*

Scenario: Write a function sumDigits(num) that adds up all the individual digits of a number.

- If the input is not a number, return "Invalid".

Expected Output:

```
sumDigits(1234) → 10    (1+2+3+4)
```

Hints:

1. Use % 10 to get the last digit, then Math.floor(num / 10) to remove it
2. Repeat in a while loop until num becomes 0

Write your code here:

B-02 — Running Total Generator 🟡 Medium

Concepts Used: *Loops · Arrays*

Scenario: ExpenseMate, a budgeting app, shows a running total next to each expense. Write a function runningTotal(amounts) that returns a new array where each element is the cumulative sum up to that point.

- Return "Invalid"
  - if input isn't an array
  - any value isn't a number

Expected Output:

```
runningTotal([100, 50, 25]) → [100, 150, 175]
```

Hints:

1. Keep a variable that tracks the running sum so far
2. For each amount, add it to the running sum and push the new sum into a result array

Write your code here:

B-03 — Reverse Each Word in a Sentence 🔴 Hard

Concepts Used: *Strings · Arrays · Loops*

Scenario: Write a function reverseEachWord(sentence) that reverses every individual word but keeps the word order the same.

- If the input is not a string, return "Invalid".

Expected Output:

```
reverseEachWord('Hero is strong') → 'oreH si gnorts'
```

Hints:

1. split(' ') into an array of words
2. Reverse each word individually, then join(' ') them back

Write your code here:

B-04 — Second Largest Number 🟡 Medium

Concepts Used: *Arrays · Loops · Conditionals*

Scenario: Write a function secondLargest(arr) that finds the second largest number in an array WITHOUT using sort().

- If the input is not an array, return "Invalid".
- If any element is not a number, return "Invalid".

Expected Output:

```
secondLargest([45, 12, 89, 3, 67]) → 67
```

Hints:

1. Track both the largest AND second largest as you loop
2. When you find a new largest, the old largest becomes the new second largest

Write your code here:

B-05 — Count Even & Odd in an Array 🟡 Medium

Concepts Used: *Arrays · Objects · Functions · Loops*

Scenario: Write a function countEvenOdd(arr) that returns an object like { even: 3, odd: 2 } counting how many even and odd numbers are in the array.

- If the input isn't an array, return "Invalid".
- If the array contains any non-number values, return "Invalid".

Expected Output:

```
countEvenOdd([1,2,3,4,5]) → { even: 2, odd: 3 }
```

Hints:

1. Start with an object { even: 0, odd: 0 }
2. Inside the loop, increment result.even or result.odd based on % 2

Write your code here:

B-06 — Employee Salary Slip 🟡 Medium

Concepts Used: *Objects · Functions · Arithmetic*

Scenario: Given a single employee object (name, basicSalary, bonus, tax), write a function generateSalarySlip(employee) that returns a NEW object containing the employee's name and their netSalary (basicSalary + bonus - tax).

- Return "Invalid" if:

- basicSalary is not a number
- bonus is not a number
- tax is not a number

Expected Output:

```
employee = { name:'Karim', basicSalary:30000, bonus:5000, tax:2000 }
Slip: { name:'Karim', netSalary:33000 }
```

Hints:

1. netSalary = basicSalary + bonus - tax
2. Return a new object — { name: employee.name, netSalary: ... }

Write your code here:

B-07 — Character Frequency Counter 🔴 Hard

Concepts Used: *Strings · Objects · Loops*

Scenario: Write a function charFrequency(str) that returns an object showing how many times each character appears in the string.

- If the input is not a string, return "Invalid".

Expected Output:

```
charFrequency('hero') → { h:1, e:1, r:1, o:1 }
charFrequency('hello') → { h:1, e:1, l:2, o:1 }
```

Hints:

1. Start with an empty object {}
2. For each character, if it's already a key, increment it — otherwise set it to 1

Write your code here:

B-08 — Cart Total Validator 🟡 Medium

Concepts Used: *Numbers · Loops · Arrays*

Scenario: QuickCart, an e-commerce app, double-checks that the sum of individual item prices matches the total shown at checkout. Write a function verifyCartTotal(itemPrices, displayedTotal) that returns true if they match.

- Return "Invalid" if:
  - itemPrices is not an array
  - displayedTotal is not a number
  - any price in itemPrices is not a number

Expected Output:

```
verifyCartTotal([250, 400, 150], 800) → true
verifyCartTotal([250, 400, 150], 750) → false
```

Hints:

1. Loop through itemPrices and add them all up
2. Compare the computed sum to displayedTotal with ===

Write your code here:

Part 3 — Debugging

Every programmer writes bugs — the skill isn't avoiding them, it's finding them fast. There are three categories of bugs, and recognizing which type you're facing tells you where to look.

Error Type	What it means	Example
Syntax Error	Code breaks JS grammar rules — won't even run	Missing bracket, missing comma
Runtime Error	Code runs, then crashes mid-way	Calling undefined variable/method
Logical Error	Code runs fine, gives WRONG answer silently	Using = instead of ===, off-by-one loop

The scariest bugs: Logical errors are the hardest to catch because JavaScript gives no error message at all — the program just quietly produces the wrong result. Always test your output against expected values.

Type A — Instructor Debugs Live

AD-01 — The Missing Comma

Error Category: Syntax Error

What happens when you run this: Uncaught SyntaxError: Unexpected identifier 'age' — the code won't even run.

Buggy Code:

```
const student = {
  name: 'Ayan'
  age: 20
};
```

Hints:

1. Compare this object literal to the correct syntax from Session 2
2. Every property except the last one needs something after its value

Your fix:

AD-02 — The Typo Trap

Error Category: Runtime Error

What happens when you run this: Uncaught ReferenceError: nmae is not defined

Buggy Code:

```
const name = 'Sakib';
console.log(nmae);
```

Hints:

1. Read the variable name character by character
2. JavaScript is case AND spelling sensitive — it won't guess what you meant

Your fix:

AD-03 — The Silent Assignment

Error Category: Logical Error

What happens when you run this: Always prints 'Perfect score!' no matter what score actually is — AND score gets overwritten to 100.

Buggy Code:

```
let score = 85;

if (score = 100) {
  console.log('Perfect score!');
} else {
  console.log('Score:', score);
}
```

Hints:

1. Count the equals signs in the if condition
2. = assigns a value. Which operator COMPARES two values?

Your fix:

AD-04 — The Unreachable Line

Error Category: Logical Error

What happens when you run this: Nothing gets printed to the console at all — the log line seems to just... not exist.

Buggy Code:

```
function getDouble(num) {
  return num * 2;
  console.log('Function finished');
}

getDouble(5);
```

Hints:

1. What does return do the instant it runs?
2. Is there any way for code physically below a return statement to execute?

Your fix:

Type B — Students Debug

BD-01 — The Missing Parenthesis

Error Category: Syntax Error

What happens when you run this: Uncaught SyntaxError: missing) after argument list

Buggy Code:

```
function greet(name {
  console.log('Hi ' + name);
}
```

Hints:

1. Look carefully at the function's parameter list
2. Every (needs a matching)

Your fix:

BD-02 — The Method That Isn't There

Error Category: Runtime Error

What happens when you run this: Prints undefined instead of 2 — no crash, but the value is wrong.

Buggy Code:

```
const fruits = ['apple', 'banana'];
console.log(fruits.lenght);
```

Hints:

1. This is a spelling mistake, not a real JavaScript keyword
2. The correct property has a very common English word spelled correctly

Your fix:

BD-03 — The Extra Loop

Error Category: Logical Error

What happens when you run this: Prints 10, 20, 30, undefined — one extra line that shouldn't be there.

Buggy Code:

```
const arr = [10, 20, 30];
for (let i = 0; i <= arr.length; i++) {
  console.log(arr[i]);
}
```

Hints:

1. What is arr.length for a 3-item array?
2. Compare <= vs < — which one stops at the correct last index?

Your fix:

BD-04 — The Reversed Discount

Error Category: Logical Error

What happens when you run this: applyDiscount(1000, 20) returns 1200 instead of a smaller discounted price of 800.

Buggy Code:

```
function applyDiscount(price, percent) {
  return price + (price * percent / 100);
}

console.log(applyDiscount(1000, 20));
// Expected a DISCOUNT, got a price INCREASE
```

Hints:

1. A discount should REDUCE the price, not increase it
2. Which arithmetic operator subtracts instead of adds?

Your fix: